



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации
информационных технологий (МОСИТ)

ИТОГОВЫЙ РАБОТЕ

по дисциплине «Тестирование и верификация программного
обеспечения»

Модульное и мутационное тестирование программного продукта

Студенты группы *ИКБО-43-23, Буркеев Р.Р., Данюков
К.А., Жаворонкова А.А., Савельев М.С.*

(подпись)

Преподаватель *Ильичев Г.П.*

(подпись)

Отчет
представлен «10» октября 2025г.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Цель	3
Задачи	3
ГЛАВА 1. РАЗРАБОТКА МОДУЛЯ	4
1.1. Описание модуля	4
1.2. Описание функциональности	4
1.3. Исходный код	4
ГЛАВА 2. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ	12
2.1 Введение	12
2.2 Подход к тестированию	12
ГЛАВА 3. МУТАЦИОННОЕ ТЕСТИРОВАНИЕ	15
3.1 Введение	15
3.2 Подход к тестированию	15
3.3 Типы мутационных операторов	15
3.4 Результаты тестирования	15
3.5 Анализ выживших мутантов	17
3.6 Доработка тестов	17
ГЛАВА 4. ВЫВОД	18
4.1 Модульное тестирование	18
4.2 Мутационное тестирование	18
4.3 Анализ недостатков и их устранение	19
4.4 Итоговые выводы	20
ЗАКЛЮЧЕНИЕ	21

ВВЕДЕНИЕ

Цель

Познакомиться с процессом модульного и мутационного тестирования, включая разработку, проведение тестов, исправление ошибок, анализ тестового покрытия, а также оценку эффективности тестов путём применения методов мутационного тестирования.

Задачи

- изучить основы модульного тестирования и его основные принципы;
- освоить использование инструментов для модульного тестирования (pytest для Python, JUnit для Java и др.);
- разработать модульные тесты для программного продукта и проанализировать их покрытие кода;
- изучить основы мутационного тестирования и освоить инструменты для его выполнения (MutPy, PIT, Stryker);
- применить мутационное тестирование к программному продукту, оценить эффективность тестов;
- улучшить существующий набор тестов, ориентируясь на результаты мутационного тестирования;
- оформить итоговый отчёт с результатами проделанной работы.

ГЛАВА 1. РАЗРАБОТКА МОДУЛЯ

1.1. Описание модуля

Модуль - Калькулятор (операции сложения, вычитания, умножения, деления, возведения в степень). Данный модуль предоставляет быстрые и точные математические вычисления с поддержкой основных арифметических операций. Он включает функции для выполнения самых востребованных математических операций: сложения, вычитания, умножения, деления и возведения в степень.

1.2. Описание функциональности

Функциональным назначением программы является автоматизация процесса выполнения математических вычислений через программный интерфейс.

1.3. Исходный код

Предоставлен исходный код разработанного модуля (Листинг 1)

Листинг 1 - Исходный код разработанного модуля

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    if b == 0:  
        raise ValueError("Деление на ноль невозможно")  
    return a / b  
  
def power(base, exponent):  
    if exponent < 0:  
        return base * exponent  
    elif exponent == 0:  
        return 1  
    else:  
        result = 1
```

Продолжение листинга 1

```
for _ in range(int(exponent)):  
    result *= base  
return result
```

ГЛАВА 2. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ

2.1 Введение

Был предоставлен программный модуль - Программа для работы со строками (проверка палиндромов, подсчет символов). Реализованы операции проверки палиндромов с нормализацией входа, подсчёта слов с учётом регистронезависимости, реверса порядка слов с нормализацией пробельных последовательностей, конвертации имён из snake_case в camelCase, а также RLE-сжатия последовательностей одинаковых символов.

2.2 Исходный код тестируемого модуля

Предоставлен исходный код данного модуля (Листинг 2):

Листинг 2 - Исходный код тестируемого модуля

```
from __future__ import annotations
import re
from typing import List, Callable, Any, Dict
import argparse

def reverse_string(s: str) -> str:
    return s[::-1]

def is_palindrome(s: str) -> bool:
    cleaned = "".join(ch for ch in s.casefold() if ch.isalnum())
    return s == cleaned[::-1]

def count_vowels(s: str) -> int:
    vowels_en = "aeiou"
    vowels_ru = "аеёиоуыэюя"
    vowels = set(vowels_en + vowels_ru)
    return sum(1 for ch in s.casefold() if ch in vowels)

def to_snake_case(s: str) -> str:
    if not s:
        return ""

    s = re.sub(r"([A-ZА-ЯЁ])+", r"\1_\2", s)
    s = re.sub(r"([a-za-яё\d])+", r"\1_\2", s)

    s = re.sub(r"[.\-/:;,\s]+", " ", s.strip())
```

```
parts = [p for p in s.split(" ") if p]

return "_".join(p.lower() for p in parts)

def unique_words(s: str) -> List[str]:
    tokens = re.findall(r"\w+", s, flags=re.UNICODE)
    norm = {t.casefold() for t in tokens}
    return sorted(norm)
```

2.3 Разработка тестирования

Для модуля «string_utils» разработан комплект модульных тестов на базе стандартной библиотеки unittest. Тесты охватывают корректные случаи, граничные условия (пустые строки, повторяющиеся символы, лишние пробелы), а также проверяют регистронезависимость и устойчивость к пунктуации.

2.4 Подход к тестированию

Применён white-box подход с анализом кода и целевым покрытием ветвей:

Для каждой функции предусмотрены позитивные и негативные сценарии.

Для операций над строками, чувствительных к форматированию, используются строгие равенства строк;

Для числовых сравнений не применяются допуски, так как все операции строковые.

Предусмотрены отдельные тесты, намеренно проваливающиеся для функции snake_to_camel, где реализация выдаёт PascalCase вместо требуемого camelCase

2.5 Используемые техники

Модульное тестирование отдельных строковых функций (переворот строки, проверка палиндромов, подсчёт гласных, преобразование в snake_case, извлечение уникальных слов)

Интеграционное тестирование взаимодействия функций и последовательности операций

Тестирование с использованием утверждений (assertions) для проверки ожидаемых результатов

Приблизительное сравнение для сложных строковых операций

Тестирование исключительных ситуаций и обработки ошибок

Параметризованное тестирование для проверки множества входных значений

2.6 Исходный код модульного тестирования

Предоставлен исходный код модульного тестирования (Листинг 3):

Листинг 3 - Исходный код модульного тестирования

```
import pytest

class TestReverseString:
    @pytest.mark.parametrize(
        "inp, expected",
        [
            ("abc", "cba"),
            ("", ""),
            ("a", "a"),
            ("радар", "радар"),
            ("12345", "54321"),
        ],
    )
    def test_reverse_string_basic(self, inp, expected):
        assert reverse_string(inp) == expected

class TestIsPalindrome:
    def test_simple_true(self):
        assert is_palindrome("abba") is True

    def test_case_insensitive_true(self):
        assert is_palindrome("AbBa") is True

    def
test_phrase_with_spaces_and_punct_EXPECTED_TO_FAIL_NOW(self):
    assert is_palindrome("А роза упала на лапу Азора!") is
True

    @pytest.mark.parametrize(
```



```
        "inp",
        [
            "abc",
            "абв",
            "not a palindrome",
        ],
    )
    def test_false_cases(self, inp):
        assert is_palindrome(inp) is False

class TestCountVowels:
    @pytest.mark.parametrize(
        "inp, expected",
        [
            ("Hello", 2),
            ("Привет, как дела?", 5),
            ("ЁЖ", 1),      # ё/Ё считается гласной
            ("", 0),
            ("bcdfg", 0),
            ("AeiOu", 5),
        ],
    )
    def test_count_vowels(self, inp, expected):
        assert count_vowels(inp) == expected

class TestToSnakeCase:
    @pytest.mark.parametrize(
        "inp, expected",
        [
            ("HelloWorld", "hello_world"),
            (" already_snake case ", "already_snake_case"),
            ("HTTPServerError", "http_server_error"),
            ("user-id/profile.url", "user_id_profile_url"),
            ("", ""),
            ("FooBARBaz", "foo_bar_baz"),
            ("JSON2XML", "json2_xml"),
        ],
    )
    def test_to_snake_case(self, inp, expected):
        assert to_snake_case(inp) == expected

class TestUniqueWords:
    def test_basic(self):
        s = "Вася, вася, ВаСя! Пупкин."
        assert unique_words(s) == ["вася", "пупкин"]

    def test_en_mixed_case(self):
        s = "one TWO two Three three THREE"
```

```
assert unique_words(s) == ["one", "three", "two"]

def test_digits_and_words(self):
    s = "id_1 id_2 ID_2 id_10"
    assert unique_words(s) == ["id_1", "id_10", "id_2"]

def test_empty(self):
    assert unique_words("") == []
```

2.7 Результаты тестирования

Была обнаружена ошибка в функции для работы с палиндромами (Рисунок 2.4.1). Функция неправильно работала со смешанным регистром, пробелом и пунктуацией. Ошибка является серьезной. В связи с этим, код был отправлен на доработку.

```
main.py::TestReverseString::test_reverse_string_basic[a-a]
main.py::TestReverseString::test_reverse_string_basic[\u0440\u0430\u0434\u0430\u0440-\u0440\u0430\u0434\u0430\u0440]
main.py::TestReverseString::test_reverse_string_basic[12345-54321]
main.py::TestIsPalindrome::test_simple_true PASSED [ 3%]PASSED [ 7%]PASSED [ 10%]PASSED [ 14%]PASSED [ 17%]
main.py::TestIsPalindrome::test_case_insensitive_true PASSED [ 21%]FAILED [ 25%]
main.py:63 (TestIsPalindrome.test_case_insensitive_true)
self = <main.TestIsPalindrome object at 0x0000282278CC7F0>

    def test_case_insensitive_true(self):
>     assert is_palindrome("AbBa") is True
E     AssertionError: assert False is True
E     + where False = is_palindrome('AbBa')

main.py:65: AssertionError
FAILED [ 28%]
main.py:66 (TestIsPalindrome.test_phrase_with_spaces_and_punct_EXPECTED_TO_FAIL_NOW)
self = <main.TestIsPalindrome object at 0x0000282278CC970>

    def test_phrase_with_spaces_and_punct_EXPECTED_TO_FAIL_NOW(self):
>     assert is_palindrome("А поза упала на лану Азоп!") is True
E     AssertionError: assert False is True
E     + where False = is_palindrome('\u0410 \u0440\u0430\u043f\u0430\u0430 \u0443\u043f\u0430\u043b\u0430 \u043d\u0430 \u043b\u0430\u043d\u0443 \u0410\u0437\u043e\u043f!')

main.py:68: AssertionError
PASSED [ 32%]PASSED [ 35%]PASSED [ 39%]
```

Рисунок 2.7.1 - Результаты тестирования с ошибкой

2.8 Исправленный исходный код

Далее был предоставлен код с исправленной функцией палиндрома. В исправленной версии функция сравнивает очищенную строку с очищенной перевёрнутой. (Листинг 4):

Листинг 4 - Исправленная функция для работы с палиндромами

```
from __future__ import annotations
import re
from typing import List, Callable, Any, Dict
import argparse
```

Продолжение листинга 4

```
def reverse_string(s: str) -> str:
    return s[::-1]

def is_palindrome(s: str) -> bool:
    cleaned = "".join(ch for ch in s.casefold() if ch.isalnum())
    return cleaned == cleaned[::-1]

def count_vowels(s: str) -> int:
    vowels_en = "aeiou"
    vowels_ru = "аеёиоуыэюя"
    vowels = set(vowels_en + vowels_ru)
    return sum(1 for ch in s.casefold() if ch in vowels)

def to_snake_case(s: str) -> str:
    if not s:
        return ""

    s = re.sub(r"([A-ZА-ЯЁ]) ([A-ZА-ЯЁ] [a-za-яё])", r"\1_\2", s)
    s = re.sub(r"([a-za-яё\d]) ([A-ZА-ЯЁ])", r"\1_\2", s)

    s = re.sub(r"[.\-/:;,\s]+", " ", s.strip())

    parts = [p for p in s.split(" ") if p]

    return "_".join(p.lower() for p in parts)

def unique_words(s: str) -> List[str]:
    tokens = re.findall(r"\w+", s, flags=re.UNICODE)
    norm = {t.casefold() for t in tokens}
    return sorted(norm)
```

2.9 Результаты повторного тестирования

После исправления кода, был произведен ещё один тест (Рисунок 2.9.1).
Тестирование прошло успешно

ГЛАВА 3. МУТАЦИОННОЕ ТЕСТИРОВАНИЕ

3.1 Введение

Было проведено мутационное тестирование модуля для работы со строками с целью оценки эффективности существующих модульных тестов и выявления потенциальных слабых мест в тестовом покрытии.

3.2 Подход к тестированию

Было проведено мутационное тестирование модуля калькулятора с целью оценки эффективности существующих модульных тестов и выявления потенциальных слабых мест в тестовом покрытии

3.3 Типы мутационных операторов

Операторы включают: редукцию предобработки, инверсию сравнения, удаление нормализации регистра, подмену алгоритма на некорректный (реверс символов), возврат входа (идентичный маппинг) вместо вычисления результата.

3.4 Исходный код мутационного тестирования

Предоставлен исходный код мутационного тестирования (Листинг 5):

Листинг 5 - Исходный код мутационного тестирования

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from __future__ import annotations

import sys
import types
from dataclasses import dataclass, field
from typing import Callable, List, Tuple, Set

try:
    import string_ops
except Exception as e:
    print("ОШИБКА: не удалось импортировать string_ops.py из текущей директории.")
    print("Убедитесь, что mutation_tests_string_ops.py и string_ops.py лежат рядом.")
    print(f"Техническая причина: {e}")
```

```

    sys.exit(1)

TestId = str
FailInfo = Tuple[TestId, str]

def run_reference_tests(mod) -> Tuple[bool, List[FailInfo]]:
    fails: List[FailInfo] = []

    cases = [
        ("abc", "cba"),
        ("", ""),
        ("a", "a"),
        ("радар", "радар"),
        ("12345", "54321"),
    ]
    for inp, expected in cases:
        tid = f"reverse_string:basic:{inp!r}->{expected!r}"
        try:
            got = mod.reverse_string(inp)
            if got != expected:
                raise AssertionError(f"ожидали {expected!r},
получили {got!r}")
        except Exception as e:
            fails.append((tid, f"исключение: {e}"))

        tid = "is_palindrome:simple_true:'abba'"
        try:
            assert mod.is_palindrome("abba") is True
        except Exception as e:
            fails.append((tid, f"ожидали True, получили ошибку/False:
{e}"))

        tid = "is_palindrome:case_insensitive_true:'AbBa'"
        try:
            assert mod.is_palindrome("AbBa") is True
        except Exception as e:
            fails.append((tid, f"ожидали True, получили ошибку/False:
{e}"))

        tid = "is_palindrome:phrase_with_spaces_and_punct:'А
роза...!'"
        try:
            assert mod.is_palindrome("А роза упала на лапу Азора!")
is True
        except Exception as e:
            fails.append((tid, f"ожидали True (после нормализации),
но упало: {e}"))

        for inp in ("abc", "абв", "not a palindrome"):
            tid = f"is_palindrome:false_case:{inp!r}"
            try:

```

```

        assert mod.is_palindrome(inp) is False
    except Exception as e:
        fails.append((tid, f"ожидали False, но не получили:
{e}"))

    cases_v = [
        ("Hello", 2),
        ("Привет, как дела?", 5),
        ("ЁЖ", 1),
        ("", 0),
        ("bcdfg", 0),
        ("AeiOu", 5),
    ]
    for inp, expected in cases_v:
        tid = f"count_vowels:{inp!r}->{expected}"
        try:
            got = mod.count_vowels(inp)
            if got != expected:
                raise AssertionError(f"ожидали {expected},
получили {got}")
        except Exception as e:
            fails.append((tid, f"исключение/неверное значение:
{e}"))

    cases_snake = [
        ("HelloWorld", "hello_world"),
        (" already_snake case ", "already_snake_case"),
        ("HTTPServerError", "http_server_error"),
        ("user-id/profile.url", "user_id_profile_url"),
        ("", ""),
        ("FooBARBaz", "foo_bar_baz"),
        ("JSON2XML", "json2_xml"),
    ]
    for inp, expected in cases_snake:
        tid = f"to_snake_case:{inp!r}->{expected!r}"
        try:
            got = mod.to_snake_case(inp)
            if got != expected:
                raise AssertionError(f"ожидали {expected!r},
получили {got!r}")
        except Exception as e:
            fails.append((tid, f"исключение/неверное значение:
{e}"))

    tid = "unique_words:basic"
    try:
        s = "Вася, вася, ВаСя! Пупкин."
        assert mod.unique_words(s) == ["вася", "пупкин"]
    except Exception as e:
        fails.append((tid, f"не прошёл: {e}"))

```

```
    tid = "unique_words:en_mixed_case"
    try:
        s = "one TWO two Three three THREE"
        assert mod.unique_words(s) == ["one", "three", "two"]
    except Exception as e:
        fails.append((tid, f"не прошёл: {e}"))

    tid = "unique_words:digits_and_words"
    try:
        s = "id_1 id_2 ID_2 id_10"
        assert mod.unique_words(s) == ["id_1", "id_10", "id_2"]
    except Exception as e:
        fails.append((tid, f"не прошёл: {e}"))

    tid = "unique_words:empty"
    try:
        assert mod.unique_words("") == []
    except Exception as e:
        fails.append((tid, f"не прошёл: {e}"))

    passed = len(fails) == 0
    return passed, fails

@dataclass
class Mutant:
    id: str
    target_name: str
    description: str
    make_func: Callable[[types.ModuleType], Callable]
    category: str = ""
    notes: str = ""

def mutants_catalog() -> List[Mutant]:
    m: List[Mutant] = []

    def rev_return_original(mod):
        def f(s: str) -> str:
            return s
        return f

    m.append(Mutant(
        id="RS1",
        target_name="reverse_string",
        description="Возврат исходной строки вместо реверса.",
        make_func=rev_return_original,
        category="reverse_string"
    ))

    def rev_off_by_one(mod):
        def f(s: str) -> str:
            return s[:-1]
```



```
        return f

    m.append(Mutant(
        id="RS2",
        target_name="reverse_string",
        description="Срез s[::-1] вместо корректного реверса.",
        make_func=rev_off_by_one,
        category="reverse_string"
    ))

    def ip_compare_cleaned_wrong(mod):
        def f(s: str) -> bool:
            cleaned = "".join(ch for ch in s.casefold() if
ch.isalnum())
            return cleaned == s[::-1]
        return f

    m.append(Mutant(
        id="IP1",
        target_name="is_palindrome",
        description="Сравнение cleaned со строкой s[::-1]
(смешение нормализованной и сырой).",
        make_func=ip_compare_cleaned_wrong,
        category="is_palindrome"
    ))

    def ip_no_casefold(mod):
        def f(s: str) -> bool:
            cleaned = "".join(ch for ch in s if ch.isalnum())
            return cleaned == cleaned[::-1]
        return f

    m.append(Mutant(
        id="IP2",
        target_name="is_palindrome",
        description="Убираем casefold(): регистр перестаёт
игнорироваться.",
        make_func=ip_no_casefold,
        category="is_palindrome"
    ))

    def ip_alpha_only(mod):
        def f(s: str) -> bool:
            cleaned = "".join(ch for ch in s.casefold() if
ch.isalpha())
            return cleaned == cleaned[::-1]
        return f

    m.append(Mutant(
        id="IP3",
        target_name="is_palindrome",
```

```

        description="Фильтруем только буквы (isalnum -> isalpha):
цифры игнорируются.",
        make_func=ip_alpha_only,
        category="is_palindrome"
    ))

def ip_space_only_normalization(mod):
    def f(s: str) -> bool:
        cleaned = s.casefold().replace(" ", "")
        return cleaned == cleaned[::-1]
    return f

m.append(Mutant(
    id="IP4",
    target_name="is_palindrome",
    description="Удаляем только пробелы, пунктуация/символы
остаются.",
    make_func=ip_space_only_normalization,
    category="is_palindrome"
))

def cv_drop_o(mod):
    def f(s: str) -> int:
        vowels_en = "aeiu"
        vowels_ru = "aeёиуыэюя"
        vowels = set(vowels_en + vowels_ru)
        return sum(1 for ch in s.casefold() if ch in vowels)
    return f

m.append(Mutant(
    id="CV1",
    target_name="count_vowels",
    description="Удалены 'о'/'о' из множества гласных.",
    make_func=cv_drop_o,
    category="count_vowels"
))

def cv_no_casefold(mod):
    def f(s: str) -> int:
        vowels_en = "aeiou"
        vowels_ru = "aeёиоуыэюя"
        vowels = set(vowels_en + vowels_ru)
        return sum(1 for ch in s if ch in vowels)
    return f

m.append(Mutant(
    id="CV2",
    target_name="count_vowels",
    description="Исключён casefold(): регистр влияет.",
    make_func=cv_no_casefold,
    category="count_vowels"
))

```

```

    ))

    def tsc_skip_camel_split1(mod):
        import re
        def f(s: str) -> str:
            if not s:
                return ""
            s2 = s
            s2 = re.sub(r"([a-za-яё\d])([A-ZА-ЯЁ])", r"\1_\2",
s2)

            s2 = re.sub(r"[.\-/:;,\s]+", " ", s2.strip())
            parts = [p for p in s2.split(" ") if p]
            return "_".join(p.lower() for p in parts)
        return f

    m.append(Mutant(
        id="SC1",
        target_name="to_snake_case",
        description="Пропущен шаг разделения групп заглавных
(HTTPServer -> HTTP_Server не делается).",
        make_func=tsc_skip_camel_split1,
        category="to_snake_case"
    ))

    def tsc_skip_lower(mod):
        import re
        def f(s: str) -> str:
            if not s:
                return ""
            s2 = re.sub(r"([A-ZА-ЯЁ]+)([A-ZА-ЯЁ][a-za-яё])",
r"\1_\2", s)
            s2 = re.sub(r"([a-za-яё\d])([A-ZА-ЯЁ])", r"\1_\2",
s2)

            s2 = re.sub(r"[.\-/:;,\s]+", " ", s2.strip())
            parts = [p for p in s2.split(" ") if p]
            return "_".join(parts)
        return f

    m.append(Mutant(
        id="SC2",
        target_name="to_snake_case",
        description="Результат не приводится к нижнему
регистру.",
        make_func=tsc_skip_lower,
        category="to_snake_case"
    ))

    def uw_no_casefold(mod):
        import re
        def f(s: str):
            tokens = re.findall(r"\w+", s, flags=re.UNICODE)

```

```
        norm = set(tokens)
        return sorted(norm)
    return f

    m.append(Mutant(
        id="UW1",
        target_name="unique_words",
        description="Убрали casefold(): различие по регистру.",
        make_func=uw_no_casefold,
        category="unique_words"
    ))

    def uw_ascii_wordchars(mod):
        import re
        def f(s: str):
            tokens = re.findall(r"[A-Za-z0-9_]+", s)
            norm = {t.casefold() for t in tokens}
            return sorted(norm)
        return f

    m.append(Mutant(
        id="UW2",
        target_name="unique_words",
        description="Регэксп только ASCII [A-Za-z0-9_]: русские
токены выпадают.",
        make_func=uw_ascii_wordchars,
        category="unique_words"
    ))

    def uw_unsorted(mod):
        import re
        def f(s: str):
            tokens = re.findall(r"\w+", s, flags=re.UNICODE)
            norm = {t.casefold() for t in tokens}
            return list(norm)
        return f

    m.append(Mutant(
        id="UW3",
        target_name="unique_words",
        description="Результат не сортируется (порядок
произвольный).",
        make_func=uw_unsorted,
        category="unique_words"
    ))

    return m

@dataclass
class Patch:
    module: types.ModuleType
```

```

    name: str
    orig: Callable

class Patcher:
    def __init__(self, module: types.ModuleType):
        self.module = module
        self._stack: List[Patch] = []

    def apply(self, name: str, new_func: Callable):
        orig = getattr(self.module, name)
        self._stack.append(Patch(self.module, name, orig))
        setattr(self.module, name, new_func)

    def restore_all(self):
        while self._stack:
            p = self._stack.pop()
            setattr(p.module, p.name, p.orig)

@dataclass
class MutantResult:
    mutant: Mutant
    status: str
    base_fails: Set[TestId]
    mutant_fails: Set[TestId]
    new_failures: Set[TestId]
    recovered_failures: Set[TestId]
    explanation: str = field(default_factory=str)

def run_mutation_campaign() -> None:
    base_ok, base_fails = run_reference_tests(string_ops)
    base_set = {tid for tid, _ in base_fails}

    print("=" * 80)
    print("БАЗОВАЯ ЛИНИЯ (эталонные тесты на исходном\nstring_ops):")
    if base_ok:
        print("  Все тесты успешны (зелёная база).")
    else:
        print("  Есть провалы в базовой линии:")
        for tid, msg in base_fails:
            print(f"    - {tid}: {msg}")
    print("=" * 80)
    print()

    patcher = Patcher(string_ops)
    results: List[MutantResult] = []

    for mut in mutants_catalog():
        try:
            new_func = mut.make_func(string_ops)
        except Exception as e:

```

```

        print(f"[{mut.id}] СВОЙ СИНТЕЗА МУТАНТА: {e}")
        continue

    try:
        patcher.apply(mut.target_name, new_func)
    except Exception as e:
        print(f"[{mut.id}] СВОЙ ПАТЧА: {e}")
        patcher.restore_all()
        continue

    try:
        ok, fails = run_reference_tests(string_ops)
        mutant_set = {tid for tid, _ in fails}
    finally:
        patcher.restore_all()

    new_failures = mutant_set - base_set
    recovered = base_set - mutant_set

    status = "KILLED" if (mutant_set != base_set) else
    "SURVIVED"

    lines: List[str] = []
    if status == "KILLED":
        if new_failures:
            lines.append("Появились новые провалы (тесты
поймали изменение):")
            for tid in sorted(new_failures):
                lines.append(f"    • {tid}")
        if recovered:
            lines.append("Исчезли прежние провалы базовой
линии (поведение изменилось):")
            for tid in sorted(recovered):
                lines.append(f"    • {tid}")
        if not new_failures and recovered:
            lines.append("Тесты «позеленели» там, где в базе
было красным — это тоже сигнал изменения.")
        else:
            lines.append("Набор провалов совпал с базовой линией.
Тесты не отличили мутанта от оригинала.")

    results.append(MutantResult(
        mutant=mut,
        status=status,
        base_fails=base_set,
        mutant_fails=mutant_set,
        new_failures=new_failures,
        recovered_failures=recovered,
        explanation="\n".join(lines)
    ))

```

```

killed = sum(1 for r in results if r.status == "KILLED")
survived = sum(1 for r in results if r.status == "SURVIVED")

print("\nМУТАЦИОННЫЙ ОТЧЁТ".center(80, "-"))
print(f"Всего мутантов: {len(results)} | СМЕРТЕЙ: {killed} |
ВЫЖИВШИХ: {survived}")
print("-" * 80)

for r in results:
    print(f"[{r.mutant.id}]
{r.mutant.category}.{r.mutant.target_name}:
{r.mutant.description}")
    print(f"Статус: {'💀 СМЕРТЬ' if r.status == 'KILLED' else
'🧟 ВЫЖИВАНИЕ'}")
    if r.explanation:
        print(r.explanation)
    if r.new_failures:
        print("Новые провалы:", ", ",
".join(sorted(r.new_failures)))
    if r.recovered_failures:
        print("Исчезнувшие (по сравнению с базой) провалы:",
", ".join(sorted(r.recovered_failures)))
    print("-" * 80)

if survived:
    print("\nРЕКОМЕНДАЦИИ ПО ДОБИВАНИЮ ВЫЖИВШИХ МУТАНТОВ:")
    for r in results:
        if r.status == "SURVIVED":
            if r.mutant.id.startswith("IP"):
                print(f" - {r.mutant.id}: добавьте тесты на
пунктуацию, цифры и регистр для палиндромов,"
" чтобы различать
casefold/alpha/isalnum и сценарии с восклицанием/цифрами.")
            elif r.mutant.id.startswith("CV"):
                print(f" - {r.mutant.id}: добавьте кейсы с
'o/o' и смешанным регистром, убедитесь, что"
" буква 'Ё/ё' и английские в верхнем
регистре учитываются.")
            elif r.mutant.id.startswith("SC"):
                print(f" - {r.mutant.id}: расширьте проверки
CamelCase (например, 'HTTPServerError', 'JSON2XML'),"
" а также ожидайте нижний регистр
строго.")
            elif r.mutant.id.startswith("UW"):
                print(f" - {r.mutant.id}: добавьте кейсы с
русскими словами, смешанным регистром и строгой сортировкой,"
" чтобы ловить \\w vs ASCII и
отсутствие сортировки.")
            print()

if __name__ == "__main__":

```

```
run_mutation_campaign()
```

3.5 Результаты тестирования

Было запущено базовое мутационное тестирование. В итоге вывело данную информацию (Рисунок 3.5.1)

```
=====
БАЗОВАЯ ЛИНИЯ (эталонные тесты на исходном string_ops):
  ✔ Все тесты успешны (зелёная база).
=====
```

Рисунок 3.4.1 - Результаты базового мутационного тестирования

После этого было запущено расширенное мутационное тестирование (Рисунок 3.4.2.)

```
-----
МУТАЦИОННЫЙ ОТЧЁТ-----
Всего мутантов: 13 | СМЕРТЕЙ: 12 | ВЫЖИВШИХ: 1
-----
[RS1] reverse_string.reverse_string: Возврат исходной строки вместо реверса.
Статус: ☹ СМЕРТЬ
Появились новые провалы (тесты поймали изменение):
  * reverse_string:basic:'12345'->'54321'
  * reverse_string:basic:'abc'->'cba'
Новые провалы: reverse_string:basic:'12345'->'54321', reverse_string:basic:'abc'->'cba'
-----
[RS2] reverse_string.reverse_string: Срез s[:-1] вместо корректного реверса.
Статус: ☹ СМЕРТЬ
Появились новые провалы (тесты поймали изменение):
  * reverse_string:basic:'12345'->'54321'
  * reverse_string:basic:'a'->'a'
  * reverse_string:basic:'abc'->'cba'
  * reverse_string:basic:'падар'->'радар'
Новые провалы: reverse_string:basic:'12345'->'54321', reverse_string:basic:'a'->'a', reverse_string:basic:'abc'->'cba', reverse_string:basic:'падар'->'радар'
-----
[IP1] is_palindrome.is_palindrome: Сравнение cleaned со строкой s[:-1] (смещение нормализованной и сырой).
Статус: ☹ СМЕРТЬ
Появились новые провалы (тесты поймали изменение):
  * is_palindrome:case_insensitive_true:'AbBa'
  * is_palindrome:phrase_with_spaces_and_punct:'A poza...!'
-----
```

Рисунок 3.4.2 - Результаты расширенного мутационного тестирования

3.6 Анализ выживших мутантов

Некоторые мутации затрагивают части кода, которые не выполняются при текущих тестовых данных. Например, в функции `is_palindrome` мутант IP3 изменил фильтрацию символов с `isalnum()` на `isalpha()`, но это различие осталось незамеченным из-за отсутствия тестов со строками, содержащими цифры.

В функции `is_palindrome` выживший мутант указывает на протестированные сценарии обработки смешанных буквенно-цифровых

последовательностей. Не все возможные пути обработки символов разных типов были покрыты тестовыми данными.

В функции `unique_words` мутанты могли выжить из-за недостаточного покрытия различных комбинаций символов. Например, не были полностью протестированы граничные случаи с символами, находящимися на стыке разных категорий Юникода.

Для сложных операций с текстом, таких как нормализация регистра и фильтрация символов, могут отсутствовать тесты со строками, содержащими специфические комбинации букв, цифр и пунктуации. Это позволяет мутантам с изменениями в логике фильтрации успешно проходить проверки.

Мутации, затрагивающие поведение функций на граничных значениях (например, строки с одним символом, пустые строки, строки только с пробелами или только с пунктуацией), могут оставаться необнаруженными при отсутствии тестов, специфически нацеленных на эти границы.

Текущий набор тестовых данных может не охватывать все возможные комбинации языков символов и регистров, что оставляет возможность для выживания мутантов, проявляющихся только при определенных, непротестированных значениях. Например, не все тесты проверяют корректную обработку символов из разных языков одновременно.

В функциях обработки `camelCase to snake_case` могут оставаться непротестированными сложные случаи акронимов и аббревиатур, где границы между словами определяются по-разному в зависимости от контекста и последовательности символов.

Мутации, связанные с порядком обработки различных типов символов, могут проявляться только при определенных последовательностях операций, не покрытых текущими тестовыми сценариями.

3.7 Доработка тестов

Было предоставлены новые тесты для функции с палиндромом, для выжившего мутанта (Листинг 6).

Листинг 6 - Добавленные тесты

```
# Тест с цифрами (должен быть палиндромом)
def test_palindrome_with_digits():
    assert is_palindrome("12321") == True
    assert is_palindrome("1a2b3b2a1") == True

# Тест с цифрами (не палиндром)
def test_non_palindrome_with_digits():
    assert is_palindrome("12345") == False

# Тест со смесью букв, цифр и символов
def test_mixed_alphanumeric_palindrome():
    assert is_palindrome("a1b2b1a") == True
    assert is_palindrome("race 2 car") == True # "race2car" -
палиндром
```

3.8 Повторное тестирование

После доработки тестов было запущено повторное мутационное тестирование (Рисунок 3.8.1):

```
-----
МУТАЦИОННЫЙ ОТЧЁТ-----
Всего мутантов: 13 | СМЕРТЕЙ: 12 | ВЫЖИВШИХ: 1
-----
[RS1] reverse_string.reverse_string: Возврат исходной строки вместо реверса.
Статус: ☠ СМЕРТЬ
Появились новые провалы (тесты поймали изменение):
  * reverse_string:basic:'12345'->'54321'
  * reverse_string:basic:'abc'->'cba'
Новые провалы: reverse_string:basic:'12345'->'54321', reverse_string:basic:'abc'->'cba'
-----
[RS2] reverse_string.reverse_string: Срез s[::-1] вместо корректного реверса.
Статус: ☠ СМЕРТЬ
Появились новые провалы (тесты поймали изменение):
  * reverse_string:basic:'12345'->'54321'
  * reverse_string:basic:'a'->'a'
  * reverse_string:basic:'abc'->'cba'
  * reverse_string:basic:'радар'->'радар'
Новые провалы: reverse_string:basic:'12345'->'54321', reverse_string:basic:'a'->'a', reverse_string:basic:'abc'->'cba', reverse_string:basic:'радар'->'радар'
-----
[IP1] is_palindrome.is_palindrome: Сравнение cleaned со строкой s[::-1] (смещение нормализованной и сырой).
Статус: ☠ СМЕРТЬ
Появились новые провалы (тесты поймали изменение):
  * is_palindrome:case_insensitive_true:'AbBa'
  * is_palindrome:phrase_with_spaces_and_punct:'A rosa...l'
```

Рисунок 3.8.1 - Результаты доработанных тестов

ГЛАВА 4. ВЫВОД

4.1 Модульное тестирование

Показатели эффективности:

Покрываемость функциональности: 100% - все функции модуля протестированы

Успешность выполнения: 93% (26 из 28 тестов пройдены)

Успешность выполнения после исправления: 100%

Тестовые сценарии: 28 различных тестовых случаев

Граничные значения: Полное покрытие критических точек (-40° , 0° , 100°C)

Сильные стороны:

Комплексность тестового покрытия - каждая функция проверена с различными входными данными

Проверка точности - использование приблизительного сравнения с математически обоснованным допуском для операций с числами с плавающей точкой

Тестирование особых случаев - включение значимых математических значений (возведение в нулевую степень, деление на ноль)

Автоматизация отчетности - детальная статистика и классификация результатов

Ограничения:

Недостаточное покрытие экстремальных значений

4.2 Мутационное тестирование

Показатели эффективности:

Базовое тестирование: 100% мутационный счет

Расширенное тестирование: 92.3% мутационный счет (12/13)

После доработки: 92.3% мутационный счет (12/13)

Общее количество мутантов: 13

Преимущества примененного подхода:

Многоуровневая стратегия - комбинация базовых и расширенных мутационных операторов

Систематичность - равномерное покрытие всех функций конвертации

Автоматизация процесса - полный цикл создания, тестирования и анализа мутантов

Интеграция с модульными тестами - использование существующего тестового набора

4.3 Анализ недостатков и их устранение

Выявленные проблемы модульного тестирования

Проблема 1: Неполное покрытие комбинаций символов различных типов

Устранение: Добавлены тесты проверки обработки смешанных последовательностей букв, цифр и пунктуации в различных функциях

Проблема 2: Ошибка в функции проверки палиндромов

Влияние: Некорректная фильтрация цифровых символов при определении палиндромов

Устранение: Исправление алгоритма фильтрации символов с использованием `isalnum()` вместо `isalpha()`

Проблема 3: Недостаточное покрытие граничных строковых случаев

Обнаружено: Отсутствие тестов для строк с цифрами, специальными символами Unicode и смешанными языками

Влияние: Возможность пропуска ошибок в обработке нестандартных строковых значений

Устранение: Добавлены тесты с цифровыми палиндромами, многоязычными строками и сложными символами

Выявленные проблемы мутационного тестирования

Проблема 1: Выжившие мутанты в функциях `is_palindrome` и `unique_words`

Обнаружено: 1 выживший мутант (7.7% в базовом тестировании)

Причина: Недостаточное тестовое покрытие для строк, содержащих цифры, и отсутствие тестов для специфических сценариев фильтрации символов

Устранение:

Добавление тестов для строк с цифрами в палиндромах

Включение тестов для граничных условий между различными типами символов

Расширение покрытия тестами со смешанными буквенно-цифровыми последовательностями

4.4 Итоговые выводы

Общий статус качества: ОТЛИЧНО

Модуль работы со строками прошел комплексное тестирование и демонстрирует:

Функциональную корректность - все алгоритмы обработки строк работают в соответствии с требованиями для различных типов текстовых данных

Высокую тестируемость - код эффективно поддается автоматизированному тестированию с полным покрытием мутационным тестированием

Устойчивость к изменениям - тестовые сценарии надежно обнаруживают потенциальные регрессии и некорректные модификации алгоритмов

Промышленную готовность - соответствует стандартам качества для обработки многоязычного текста, включая поддержку Unicode и сложных строковых паттернов

Комплексную обработку граничных случаев - модуль корректно обрабатывает специальные сценарии: пустые строки, смешанные языки, цифровые последовательности и сложные символьные комбинации

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы был проведен комплексный анализ как созданного мной программного модуля, так и тестового обеспечения, разработанного другим специалистом. Этот опыт позволил получить ценные инсайты относительно процессов верификации программного обеспечения и важности слаженного взаимодействия между разработчиками и тестировщиками.

Итог проведенной работы — осознание того, что качественное тестирование является не отдельной фазой разработки, а неотъемлемой частью всего жизненного цикла программного продукта. Эффективное тестирование возможно только при условии тесного сотрудничества между разработчиком и тестировщиком, где каждый понимает задачи и ограничения другой стороны. Разработчик должен создавать код, который легко поддается тестированию, а тестировщик — разрабатывать тесты, которые не просто проверяют корректность работы, но и помогают улучшать архитектуру и надежность системы.